Zombie RL

Học tăng cường (reinforcement learning) cho kẻ địch
trong game bắn súng góc nhìn thứ nhất: PPO, curriculum và self-play

Python · pygame · Gymnasium · stable-baselines3 (PPO) · PyTorch

Môn học: REL

Nhóm thực hiện: Nguyễn Võ Trung Hưng (SE193635) · Ngô Phúc Nguyên (SE192516) · Lê Như Đạt (SE192807)

Mục lục

Phần A · Học tăng cường trong dự án

Tổng quan 3
Hình ảnh trong game 4
Bài toán RL: tác tử và môi trường 5
Định nghĩa MDP cho từng tác tử 6
Observation, action, POMDP của zombie 7
Hàm reward và reward shaping 8
Thuật toán PPO 9
Kiến trúc mạng và siêu tham số 10
Hai môi trường huấn luyện cho zombie 11
Curriculum, replay, warm-start 12
Tác tử người chơi 13
Self-play (league) 14
Giao thức đánh giá 15
Đường học (learning curve) 16
Kết quả 1: so với baseline 17
Kết quả 2: trước người chơi RL 18
Hiện tượng RL đã gặp và cách xử lý 19
Hạn chế và hướng phát triển 20
Đối chiếu khái niệm RL với dự án 21
Từ vựng RL 22
Tài liệu tham khảo 23

Phần B · Game và code

Các màn chơi 24
Bốn loại kẻ địch 25
Hai khẩu súng 26
Kiến trúc code 27
Chức năng từng file 28
Raycasting 29
Luật đánh của zombie 30
Cách chạy và gợi ý trình bày 31

Dự án làm gì và câu hỏi RL cần trả lời

Bài toán. Kẻ địch trong game thường chạy theo một thuật toán cố định (thấy người thì đi thẳng tới), nên người chơi chỉ cần lùi và bắn. Dự án này **huấn luyện chính sách (policy) của zombie và boss bằng học tăng cường**: chính sách chỉ nhận phần thưởng (reward) từ kết quả giao tranh, không được ai viết luật hành vi cho nó.

Tác tử 1: zombie / boss

Chính sách PPO chọn hướng đi và quyết định khi nào vung tay. Một policy cho mỗi màn, dùng chung cho mọi loại zombie của màn đó. Học qua 5 màn theo curriculum.

Tác tử 2: người chơi

Chính sách PPO điều khiển nhân vật bằng đúng các nút người thật có. Là đối thủ để đo độ mạnh của zombie và để zombie học chống lại (self-play).

Môi trường: game

Game góc nhìn thứ nhất tự viết bằng pygame (raycasting). Một bộ luật dùng chung cho game thật, huấn luyện và đánh giá.

Ba câu hỏi nghiên cứu

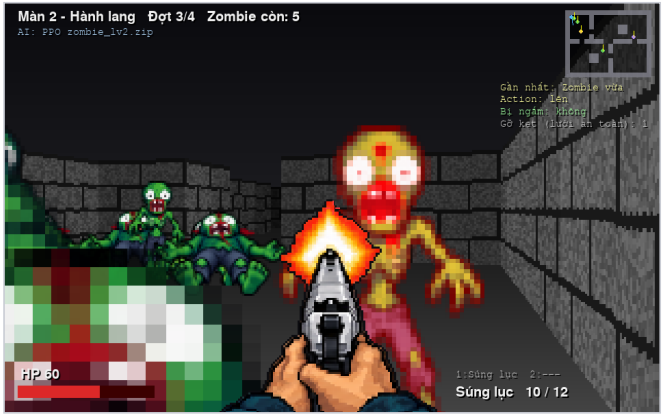
- **RQ1.** Chính sách PPO học từ reward có bắt được người chơi nhiều hơn các baseline không học (ngẫu nhiên, đi thẳng, đi theo đường ngắn nhất) không?
- **RQ2.** Khi đối thủ cũng học (self-play), chính sách zombie có thích nghi và giữ được tỉ lệ thắng không?
- **RQ3.** Thành phần thiết kế nào (hàm reward, không gian hành động, đối thủ huấn luyện) quyết định chính sách học được?

Phần RL của dự án gồm

- Mô hình hóa game thành bài toán quyết định Markov cho hai tác tử (trang 6), thiết kế observation và action (trang 7).
- Thiết kế reward, gồm reward shaping theo quãng đường đi thật và các phạt chống khai thác (trang 8).
- Thuật toán PPO, kiến trúc mạng và siêu tham số thực tế (trang 9 và 10).
- Curriculum learning, replay bản đồ cũ, warm-start khi đổi không gian hành động (trang 12).
- Self-play có cơ chế giữ bản tốt nhất (trang 14); đánh giá có đối chứng và kiểm định thống kê (trang 15 đến 18).

Hình ảnh trong game

Ảnh chụp từ game thật



Màn 2 (Hành lang): zombie vừa lao tới, súng đang nổ.



Màn 4 (Phòng thí nghiệm): zombie mập áp sát, minimap có nhiều chấm.



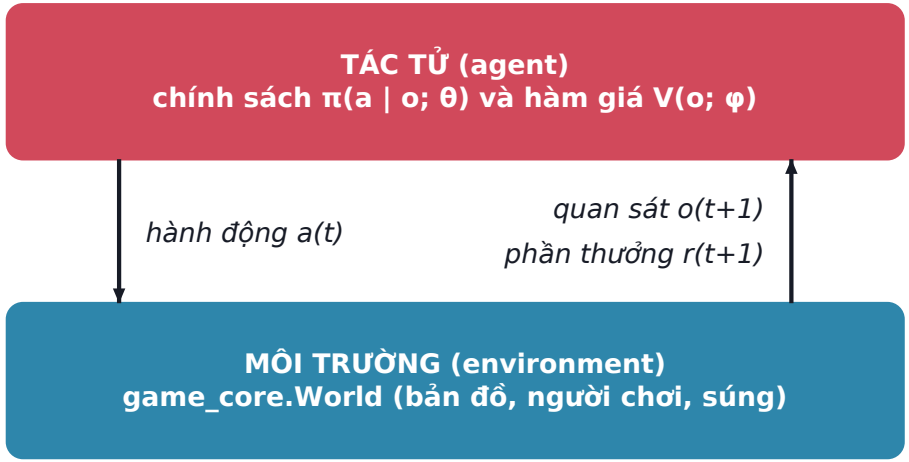
Màn 5: boss tiến vào tầm. Dòng "giữ đòn" là action PPO vừa chọn.

Điều khiển: chuột quay trái/phải · chuột trái bắn · W/S tiến lùi · A/D đi ngang · R nạp đạn · 1/2 đổi súng · Tab bảng RL · Enter sang màn · Esc thoát.

Cách đọc bảng RL (phím Tab): hiển thị đầu ra của chính sách ngay trong game

- **Minimap**: chấm màu là loại zombie; vạch vàng là hướng đi mà chính sách vừa chọn (một phần của action); vòng trắng là zombie đang bị người chơi ngắm (cờ trong observation); vòng cam là zombie đang chọn action "giữ đòn".
- **Dòng "Gần nhất / Action / Bị ngắm"**: action rời rạc (1 trong 18) mà chính sách chọn cho zombie gần người chơi nhất tại thời điểm đó.
- **Góc trên trái**: file model đang điều khiển zombie ở màn này (ví dụ *zombie_lv2.zip*). Có thể đổi sang zombie viết tay bằng ***--zombies path*** để so sánh cảm giác.

Bài toán RL: tác tử và môi trường



Mỗi bước là **Δt = 0.1 s**: tác tử chọn một action, môi trường chạy 6 khung hình game (60 FPS) với action đó rồi trả về observation và reward mới.

Ký hiệu	Ý nghĩa	Trong dự án
o_t	quan sát tại bước t	29 số (zombie), 113 số (người chơi)
a_t	hành động	1 trong 18 (zombie); 6 nhóm nút (người chơi)
r_t	phần thưởng	bảng reward, trang 8
π_θ	chính sách	mạng MLP; đầu ra là xác suất action
V_φ	hàm giá	nhánh critic của cùng thư viện PPO
γ	hệ số chiết khấu	0.99

Mục tiêu

$$J(\theta) = E_\pi \left[\sum_{t=0}^{T} \gamma^t \cdot r_t \right]$$

return kỳ vọng được chiết khấu, γ = 0.99

Hai tác tử học độc lập

- **Zombie / boss**: một policy cho mỗi màn, dùng chung cho mọi zombie của màn đó (chia sẻ tham số). Khác biệt hành vi đến từ observation (loại, vị trí, máu).
- **Người chơi**: một policy điều khiển nhân vật, dùng làm đối thủ để đo và để huấn luyện zombie.
- Khi một bên học, bên còn lại được **đóng băng** và trở thành một phần của môi trường. Nhờ vậy mỗi lần cập nhật PPO nhìn thấy môi trường đứng yên.
- Hai bên có reward đối nghịch nhau (zombie thắng khi người chơi chết và ngược lại): đây là bài toán hai tác tử cạnh tranh (xem trang 14).

Định nghĩa bài toán cho từng tác tử

Thành phần	Tác tử zombie / boss	Tác tử người chơi
Quan sát o_t	Vector 29 số thực trong [−1, 1] (trang 7)	Vector 113 số thực trong [−1, 1], tính theo hướng nhìn của người chơi (trang 13)
Không gian hành động	Discrete(18): 9 hướng đi × {cho phép đánh, giữ đòn}	MultiDiscrete([3, 3, 9, 2, 2, 2]): tiến/lùi, đi ngang, quay, bắn, nạp, đổi súng
Bước thời gian	0.1 s (6 khung hình game)	0.1 s (6 khung hình game)
Reward r_t	Thưởng tiến gần và đòn trúng, phạt bị ngắm/bị bắn/chết/vung trượt (trang 8)	Thưởng gây damage, hạ zombie, dọn màn; phạt mất máu, chết, bắn trượt (trang 13)
Kết thúc tự nhiên (terminated)	Zombie thường: đánh trúng người chơi lần đầu, hoặc bị hạ. Boss: một bên chết	Người chơi chết, hoặc dọn hết zombie của màn (màn 5: hạ boss)
Cắt do hết giờ (truncated)	400 bước = 40 s (zombie thường), 600 bước = 60 s (boss), 90 s trong game thật	240 s mỗi màn
Chiết khấu γ	0.99, tầm nhìn xa hiệu dụng ≈ 1/(1−γ) = 100 bước = 10 s	0.99, như bên trái
Tính ngẫu nhiên	Vị trí xuất phát, loại zombie, súng, nhiễu ngắm của người chơi giả; hành vi người chơi RL	Đợt zombie, vị trí, loại zombie điều khiển (PPO / path / chase), nhiễu hành vi zombie
Thuật toán	PPO (stable-baselines3)	PPO (stable-baselines3)

Ghi chú: observation là quan sát một phần của trạng thái thật (môi trường còn chứa các zombie khác, đòn đang vung, bộ đếm hồi chiêu...), nên chính xác hơn bài toán này là **POMDP**; xem phân tích ở trang 7.

Zombie nhìn gì, làm gì: observation, action, POMDP

zombie_brain.py

Nhóm	Số	Ý nghĩa
Tia nhìn tường	8	Khoảng cách tới tường theo 8 hướng quanh zombie
Hướng + khoảng cách thẳng tới người chơi	3	Vector đơn vị (x, y), độ xa
Đường đi vòng tường	3	Hướng bước kế tiếp và độ dài đường đi thật (PathFinder)
Vận tốc người chơi	2	Người chơi đang chạy hướng nào
Hướng nhìn người chơi	2	cos, sin của góc nhìn
Đang bị ngắm	1	Zombie có nằm trên đường ngắm không
Loại zombie	4	One-hot: mập, vừa, ốm, boss
Súng người chơi cầm	2	One-hot: súng lục, M4A1
Máu còn lại	1	Tỉ lệ 0–1
Người chơi nạp đạn / % đạn	2	Dấu hiệu thời cơ áp sát
Đòn của mình sẵn sàng	1	Đã hết hồi chiêu chưa

Action: 18 lựa chọn rời rạc

- 9 hướng tuyệt đối: đứng yên, 4 hướng, 4 chéo.
- × 2 chế độ: **cho phép đánh** (vung tay khi người chơi vào tầm) hoặc **giữ đòn** (chưa vung, tiếp tục di chuyển).
- Chế độ giữ đòn cho phép policy học *thời điểm* ra đòn, không chỉ hướng đi.

Chia sẻ tham số: một mạng điều khiển mọi loại zombie của màn; vector one-hot loại zombie và máu cho phép nó tách hành vi.

Phân tích: đây là POMDP, không phải MDP đầy đủ

- **Quan sát một phần**: zombie không thấy vị trí đồng đội, đòn đang vung của zombie khác, hay ý định của người chơi. Nó chỉ thấy vận tốc, hướng nhìn và cờ "đang bị ngắm" của người chơi tại thời điểm hiện tại.
- **Không có bộ nhớ**: policy là mạng truyền thẳng (MlpPolicy, không có LSTM hay xếp chồng khung hình), nên chọn action chỉ từ observation hiện tại. Đây là giả định Markov xấp xỉ; vận tốc người chơi được đưa vào observation để bù một phần thông tin về quá khứ.
- **Vì sao dùng đường đi thật thay cho bản đồ đầy đủ**: bản đồ cả màn sẽ làm observation lớn và khác kích thước giữa các màn. Hướng đi và độ dài đường đi vòng tường (PathFinder) là thông tin nén mà policy cần nhất để điều hướng.

Hàm reward và reward shaping

config.REWARD · env_zombie.py, zombie_world_env.py

Sự kiện	Giá trị	Mục đích
Rút ngắn quãng đường đi thật tới người chơi	$+0.3 \times (d_{t-1} - d_t)$	Reward dày (dense), dẫn dắt di chuyển
Áp sát người chơi (zombie thường)	+1.0, kết thúc	Mục tiêu của zombie thường
Đòn trúng người chơi (trận tay đôi, game thật)	+0.5	Thưởng đòn trúng
Hạ người chơi	+2.0, kết thúc	Mục tiêu của boss
Bị bắn trúng	−0.5 × damage / HP tối đa	Học tránh đạn
Chết	−1.0, kết thúc	Phạt chết
Đứng trong đường ngắm	−0.01 mỗi bước	Khuyến khích đổi vị trí
Vung tay trượt (whiff)	−0.05	Chống nhử đòn và vung bừa
Mỗi bước thời gian	−0.001	Ép hành động nhanh

Reward shaping

$$r_{shape} = 0.3 \cdot (d_{t-1} - d_t)$$

d = độ dài đường đi ngắn nhất (Dijkstra)

- **Tổng thưởng tiến gần trong một episode co lại thành** $0.3 \cdot (d_{đầu} - d_{cuối})$, nên đi vòng tròn không kiếm được điểm.
- Dạng này gần với potential-based shaping (Ng, Harada, Russell, 1999) với $\Phi(s) = -0.3 \cdot d(s)$, nhưng **không nhân γ**, nên không có bảo đảm lý thuyết giữ nguyên policy tối ưu. Sai lệch nhỏ vì γ = 0.99.

Vì sao cần shaping

- Reward chính (catch, hạ người chơi) **thưa**: chỉ xảy ra khi áp sát thành công. Nếu chỉ có reward này, zombie ban đầu gần như không bao giờ gặp tín hiệu để học.
- Thưởng tiến gần cho tín hiệu mỗi bước. Phần phạt (ngắm, bị bắn, vung trượt) dạy tránh và chọn thời cơ.
- **Đo khoảng cách bằng đường đi thật, không phải đường chim bay**: với khoảng cách thẳng, lao đầu vào tường vẫn "tiến gần" và được điểm (xem trang 19).

Độ lớn các thành phần: một episode zombie thường kết thúc với tối đa +1.0 (catch). Phạt bước 0.001 × 400 bước = −0.4 là tối đa; thưởng tiến gần mỗi bước ≈ 0.3 × 0.24 ô ≈ 0.07 (zombie vừa, 2.4 ô/s). Các trọng số do nhóm chọn tay, chưa có thí nghiệm bỏ từng thành phần (ablation).

Thuật toán PPO

Schulman et al. (2017), cài đặt: stable-baselines3

PPO là thuật toán **actor–critic, on-policy**: actor là policy $\pi_\theta(a \mid o)$, critic là hàm giá $V_\varphi(o)$. Policy được cập nhật bằng gradient nhưng **mức thay đổi mỗi lần bị giới hạn** để học ổn định.

$$\delta_t = r_t + \gamma\, V_\varphi(o_{t+1}) - V_\varphi(o_t) \qquad \hat{A}_t = \Sigma_{l \geq 0}\, (\gamma\lambda)^l\, \delta_{t+l}$$

ước lượng lợi thế tổng quát (GAE, Schulman et al., 2016), $\lambda = 0.95$

$$\rho_t(\theta) = \pi_\theta(a_t \mid o_t) \,/\, \pi_{\theta,\text{cũ}}(a_t \mid o_t)$$

tỉ số xác suất giữa policy mới và policy lúc thu thập dữ liệu

$$L^{\text{CLIP}}(\theta) = E_t\,[\, \min(\, \rho_t\, \hat{A}_t\, ,\, \text{clip}(\rho_t,\, 1-\varepsilon,\, 1+\varepsilon)\, \hat{A}_t\,)\,]$$

mục tiêu có cắt, $\varepsilon = 0.2$

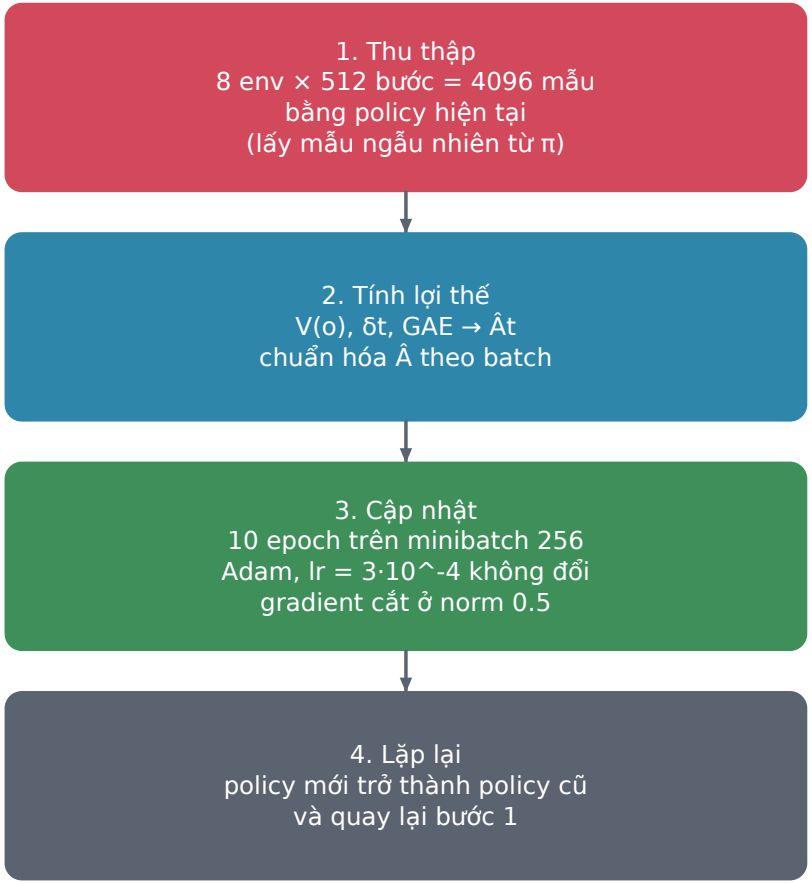
$$L = -L^{\text{CLIP}} + c_v \cdot E[(V_\varphi(o_t) - \hat{V}_t)^2] - c_e \cdot E[\, H(\pi_\theta(\cdot \mid o_t))\,]$$

loss tổng: c_v = 0.5 (critic), c_e = 0.01 (entropy)

Ý nghĩa

- **Cắt (clip)**: nếu ρ ra ngoài [0.8, 1.2] theo hướng có lợi thì gradient bằng 0, policy không bị kéo quá xa policy cũ trong một lần cập nhật.
- **Entropy**: thưởng độ ngẫu nhiên của policy để duy trì khám phá (exploration) và tránh hội tụ sớm.
- **Hết giờ (truncated)**: SB3 đặt cờ TimeLimit.truncated và bootstrap giá trị $V(o_T)$ thay vì coi là kết thúc thật.

Một vòng lặp PPO (zombie, train.py)



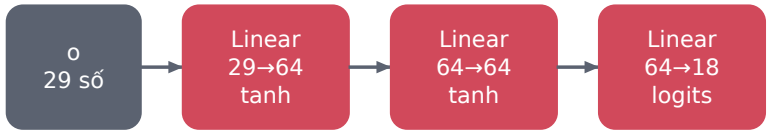
Lấy mẫu ngẫu nhiên khi huấn luyện (để khám phá), nhưng **chọn action có xác suất cao nhất (deterministic)** khi đánh giá và khi chơi thật.

Kiến trúc mạng và siêu tham số

Giá trị đọc trực tiếp từ model đã lưu và từ code

Mạng của zombie (MlpPolicy, 13 395 tham số)

policy (actor)



giá trị (critic)



Hai nhánh **không chia sẻ trọng số** (mặc định SB3). Đầu ra policy là 18 logit, qua softmax ra xác suất của 18 action.

Mạng của người chơi: 113 → 256 → 256 (tanh) với đầu policy 21 logit (3+3+9+2+2+2, mỗi nhóm một softmax) và đầu giá trị; 195 606 tham số.

Tốc độ: khoảng 2 700–3 800 bước/giây (đo trên máy huấn luyện 2 nhân CPU, 8 env, không GPU).

Phiên bản: stable-baselines3 2.9.0 · Gymnasium 1.4.0 · PyTorch 2.14.

Tham số	Zombie	Người chơi
Số env song song	8 (train.py), 4 (league.py)	4–8 (theo số nhân CPU)
n_steps mỗi env	512	1024 (khi 4 env)
Mẫu mỗi lần cập nhật	4096 (8 env), 2048 (4 env)	4096
batch_size	256	1024
n_epochs	10	10
Learning rate	3·10^-4, không đổi	3·10^-4, không đổi
γ / λ (GAE)	0.99 / 0.95	0.99 / 0.95
Clip ε	0.2	0.2
Hệ số entropy c_e	0.01	0.01
Hệ số critic c_v	0.5	0.5
Cắt gradient (norm)	0.5	0.5
Chuẩn hóa advantage	Có	Có
Tổng bước đã học	Curriculum 250k / 250k / 300k / 300k / 800k, rồi league 150k–600k mỗi màn mỗi vòng	≈ 6,0 triệu (num_timesteps = 6 008 832 trong file model)
Seed	0 (mặc định)	0 (mặc định)

Các siêu tham số không ghi tường minh trong code là mặc định của stable-baselines3 2.9.0 (đã in ra từ model để kiểm tra). Chưa chạy tìm kiếm siêu tham số.

Hai môi trường huấn luyện cho zombie

	env_zombie.py: sân tập đơn giản	zombie_world_env.py: game thật
Đối thủ	Người chơi giả viết tay (PlayerBot): có độ trễ phản xạ, nhiễu ngắm, biết vòng quanh cột (kite)	Agent người chơi PPO (mọi phiên bản trong league) và 15% người chơi viết tay. Một nửa số trận người chơi lấy mẫu action ngẫu nhiên thay vì chọn cao nhất
Ai học	Một zombie duy nhất	Một zombie "người học" (boss nếu có). Các zombie khác dùng model tốt nhất hiện tại và được đóng băng
Luật vật lý, súng, đợt zombie	Bản rút gọn: một zombie, một người chơi giả	Giống hệt game thật (game_core.World, 60 FPS, nhiều zombie, súng, nạp đạn)
Kết thúc episode	Zombie thường: đánh trúng lần đầu / bị hạ / hết 400 bước. Boss: một bên chết / hết 600 bước	Zombie người học chết / người chơi chết / hết màn / hết 90 s
Dùng trong	train.py: curriculum màn 1 → 5	league.py: mỗi vòng self-play
Mục đích	Học nhanh hành vi nền: đuổi, vòng qua tường, tránh bị ngắm	Học né, ép góc, chọn thời điểm đánh trước đối thủ biết học

Khởi tạo mỗi episode (reset)

- Vị trí xuất phát của zombie và người chơi ngẫu nhiên trên ô trống, cách nhau ít nhất 4 ô; góc nhìn ngẫu nhiên.
- Loại zombie lấy theo tỉ lệ *train_mix* của màn (ví dụ màn 1: 80% vừa, 20% mập); súng ngẫu nhiên trong các súng đã mở khóa; người chơi giả có xác suất đi thả diều theo màn.
- Ở màn k > 1, 50% episode dùng bản đồ của các màn trước (replay, trang 12).

Vì sao cần hai môi trường: môi trường đơn giản chạy nhanh (khoảng 3 000 bước/giây) nên đủ để học các hành vi cơ bản. Nhưng người chơi giả có điểm yếu cố định, nên chính sách học trên đó có thể khai thác điểm yếu đó mà không thật sự giỏi. Môi trường thứ hai dùng đối thủ biết học để kiểm tra và sửa điều này.

Curriculum learning, replay và warm-start

Màn	Tên	Quay (turn)	Phản xạ (bước)	Nhiễu ngắm	Xác suất thả diều	Loại zombie khi học	Bước train
1	Kho chứa	0.45	5	0.20	0	vừa 80%, mập 20%	250 000
2	Hành lang	0.6	4	0.14	0.25	vừa 50%, ốm 30%, mập 20%	250 000
3	Tầng hầm	0.75	3	0.09	0.5	vừa, ốm, mập mỗi loại ≈ 1/3	300 000
4	Phòng thí nghiệm	0.9	2	0.05	0.75	vừa, ốm, mập mỗi loại ≈ 1/3	300 000
5	Boss	1.0	1	0.03	1.0	chỉ boss (trận tay đôi)	800 000

*Độ khó tăng ở phía **đối thủ** (người chơi giả): phản xạ nhanh hơn, ngắm chuẩn hơn, thả diều nhiều hơn. Số liệu từ config.LEVELS.*

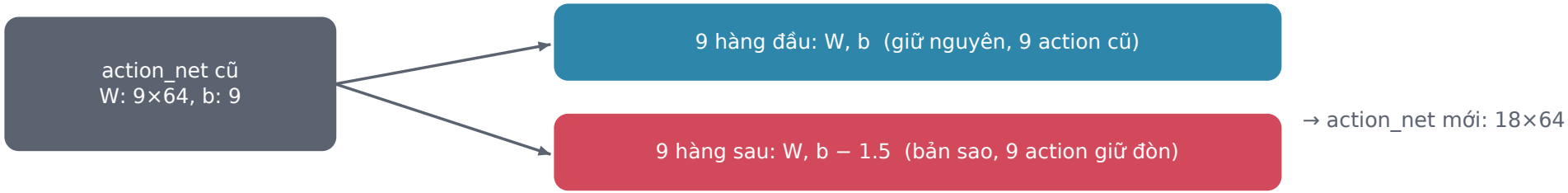
Curriculum + chuyển giao

- **Curriculum**: màn k khởi tạo từ model màn k−1 rồi học tiếp (transfer của policy), thay vì khởi tạo ngẫu nhiên. Mục tiêu: có hành vi nền (đuổi, vòng tường) trước khi gặp bài toán khó.
- **Replay**: 50% episode ở màn k dùng bản đồ các màn trước (REPLAY_PROB = 0.5), để giảm hiện tượng quên khi phân phối dữ liệu đổi.
- **Chế độ finetune** (*--finetune*): khi chỉ chỉnh nhẹ config (tốc độ, damage) thì học tiếp từ chính model của màn đó.

Warm-start khi đổi không gian hành động

- Khi thêm 9 action "giữ đòn" (9 → 18), lớp ra *action_net* đổi kích thước nên không nạp thẳng được model cũ.
- **Cách làm**: sao chép trọng số lớp ra cho 9 action cũ; 9 action mới là bản sao của 9 action cũ nhưng **bias bị trừ 1.5**. Các lớp ẩn giữ nguyên.
- Hệ quả: lúc khởi đầu mọi logit của nhóm "giữ đòn" thấp hơn đúng 1.5, nên xác suất giữ đòn ≈ $e^{-1.5}$ / (1 + $e^{-1.5}$) ≈ **18%** và policy gần như tương đương policy cũ. Phần còn lại do PPO tự học.

Lớp ra (action_net) sau warm-start



Chưa có thí nghiệm đối chứng (ablation) cho riêng curriculum hay replay trong dự án; hiệu quả của chúng chưa được đo tách biệt.

Tác tử người chơi: observation, action, reward

Nhóm observation	Số	Nội dung
Bản thân	10	Máu, đạn, đang nạp, tiến độ nạp, sẵn sàng bắn, đang đổi súng, súng 1, súng 2, đã có súng 2, số màn / 5
Tiến độ màn	3	Đợt hiện tại / tổng, số zombie còn, số đợt còn lại
Tia nhìn tường	16	Khoảng cách tới tường, tia 0 theo hướng đang nhìn
6 zombie gần nhất	6 × 14	Tồn tại, sin/cos góc tương đối, khoảng cách, có tầm nhìn thẳng, đang trong đường ngắm, loại (4), máu, đang vung tay, hướng đi (2)

Tổng 10 + 3 + 16 + 84 = **113**. Mọi hướng tính **theo hướng nhìn** của người chơi (egocentric), nên policy không cần học lại khi xoay.

Action (MultiDiscrete)

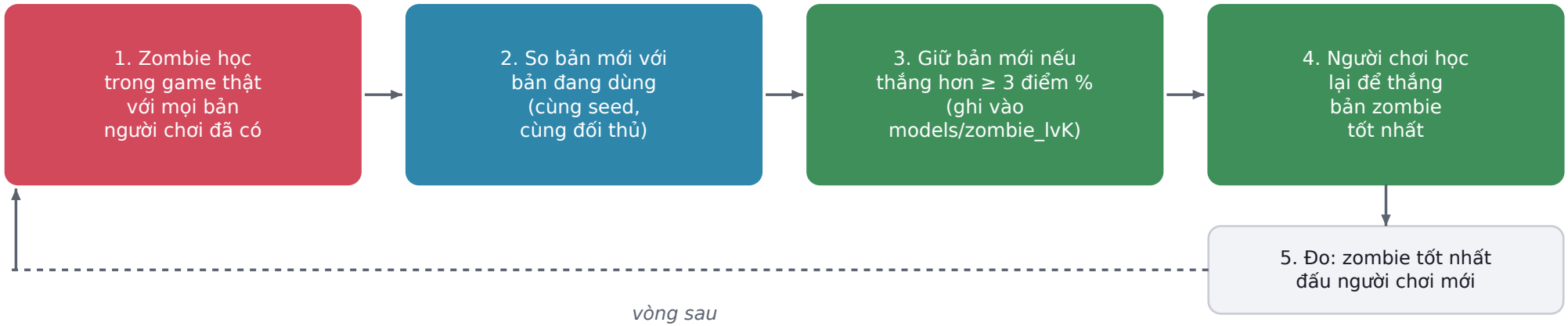
- tiến/lùi (3), đi ngang (3), quay (9 mức, tối đa 6 rad/s), bắn (2), nạp (2), đổi súng (2).
- Quyết định mỗi 0.1 s; tốc độ quay bị giới hạn để agent không quay tức thời như phần mềm.
- **Phân phối huấn luyện**: màn chọn đều ngẫu nhiên trong 5 màn; zombie đối thủ chọn đều 1/3 PPO, 1/3 đi theo đường (path), 1/3 đi thẳng (chase).

Reward người chơi	Giá trị
Gây damage	+0.5 × damage/100
Hạ zombie	+1.0
Hạ boss	+5.0
Dọn xong màn	+3.0
Mất máu	−1.0 × damage/100
Chết	−3.0
Bắn trượt	−0.005 mỗi phát
Có zombie trong đường ngắm	+0.003 mỗi bước

Vai trò trong RL: chính sách zombie được huấn luyện và đánh giá trước một đối thủ **cũng học**, không chỉ trước người chơi viết tay có điểm yếu cố định. Agent này được huấn luyện thêm khi self-play (khoảng 6,0 triệu bước tổng cộng).

Self-play: hai tác tử học đối kháng

league.py



- **Hai tác tử cạnh tranh**: reward của hai bên đối nghịch. Zombie thắng khi người chơi chết và ngược lại.
- **Đóng băng đối thủ trong mỗi pha**: nếu cả hai cùng thay đổi, môi trường nhìn từ phía mỗi bên không đứng yên (non-stationarity) và PPO, vốn giả định dữ liệu thu từ một policy cố định, dễ mất ổn định.
- **Nhóm đối thủ (pool)**: gồm mọi bản người chơi đã có (player_r00, r01, r02 được thêm dần qua các vòng) cộng người chơi viết tay; bản mới nhất xuất hiện gấp đôi. Mục đích: tránh quên cách thắng đối thủ cũ và tránh vòng lặp A thắng B, B thắng C, C thắng A.
- **Giữ bản tốt nhất**: 40 trận mỗi (màn, đối thủ) với cùng seed; bản mới chỉ thay khi hơn ≥ 0.03, vì chênh nhỏ hơn có thể chỉ do ngẫu nhiên. Ý tưởng nhóm đối thủ gần với league training trong AlphaStar (Vinyals et al., 2019), ở quy mô nhỏ hơn nhiều.

Vòng 2: % trận zombie thắng cả nhóm đối thủ (40 trận mỗi ô)

Màn	Bản đang dùng	Bản mới	Kết quả
1	0.0%	1.7%	Giữ bản cũ
2	36.7%	23.3%	Giữ bản cũ
3	0.0%	0.8%	Giữ bản cũ
4	25.8%	24.2%	Giữ bản cũ
5	85.0%	100.0%	Thay bản mới

Màn 2 và 4: bản mới kém hơn nên bị loại. Màn 5 (boss): 85.0% → 100.0%, được giữ.

Giao thức đánh giá

Phép đo	Môi trường và dữ liệu	Thống kê
1. Return và tỉ lệ bắt được so với baseline (tools/eval_current.py → eval_current.csv)	env_zombie, 100 episode mỗi màn, seed cố định 10 000 + i, policy chọn action cao nhất; so với random, chase, path	Trung bình; chưa có khoảng tin cậy riêng cho phép đo này
2. Tỉ lệ zombie thắng trước agent người chơi RL (evaluate_match.py → match_eval.csv)	game_core đầy đủ; chế độ level (100 trận mỗi màn) và campaign (100 lượt cả game, 1 mạng); cùng seed giữa các loại zombie	Khoảng tin cậy Wilson 95%; kiểm định McNemar có hiệu chỉnh liên tục trên các cặp trận cùng seed
3. Chọn bản tốt nhất và đo tiến bộ (league.py → league.csv)	40 trận mỗi (màn, đối thủ) để chọn bản; 80 trận mỗi màn để đo sau mỗi vòng, kèm mốc zombie viết tay trên cùng trận	Tỉ lệ thắng; ngưỡng giữ bản 0.03

Vì sao đánh giá như vậy

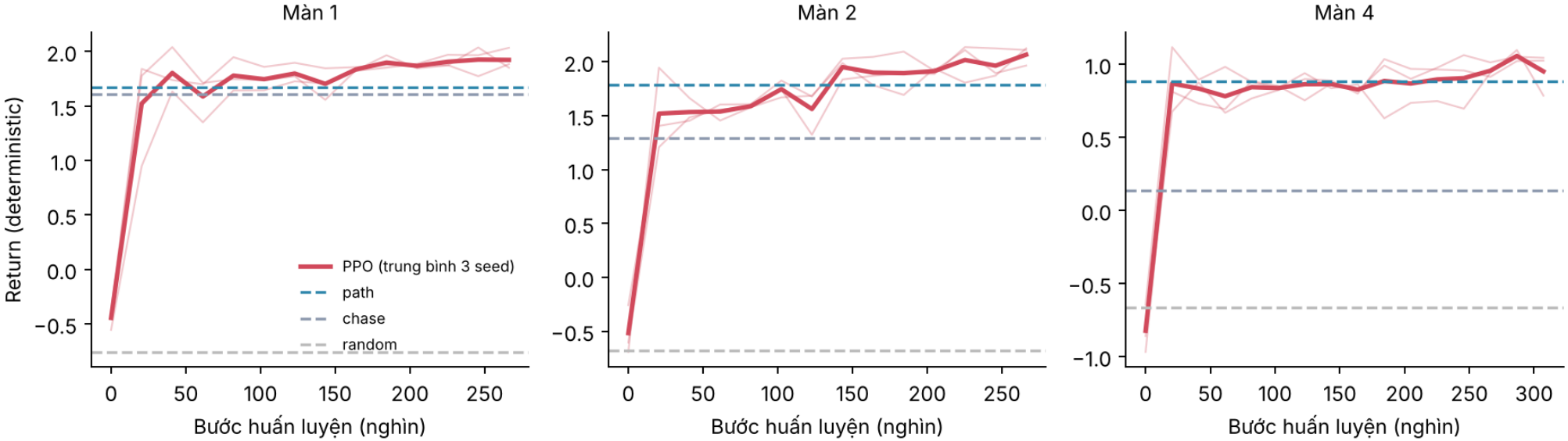
- **Return trong chính môi trường huấn luyện có thể bị khai thác** (reward hacking), nên không đủ. Tỉ lệ thắng trước một đối thủ độc lập mới đo mục tiêu thật.
- **Baseline mạnh**: zombie "đi theo đường" đã biết vòng qua tường; thắng được nó mới nói được policy học thêm điều gì đó.
- **Đánh giá deterministic**: dùng action có xác suất cao nhất, để kết quả lặp lại được và đo đúng hành vi mà game dùng khi chơi thật.

Những mối nguy đã tính tới

- **Phương sai**: 100 trận cho sai số khoảng ±10% (xem khoảng Wilson), nên chênh dưới 10 điểm khó kết luận.
- **So theo cặp**: trận thứ i của mọi loại zombie dùng cùng seed, nên khác biệt đến từ zombie, không do bản đồ hay xuất phát may rủi.
- **Đối thủ công bằng**: agent người chơi được huấn luyện đều với cả 3 loại zombie, không chỉ riêng PPO.
- **Giới hạn**: một lần huấn luyện cho mỗi cấu hình trong league (chưa lặp nhiều seed); đường học nhiều seed chỉ có cho màn 1, 2, 4 (trang 16).

Đường học (learning curve)

Huấn luyện lại từ đầu, 3 seed mỗi màn · tools/learning_curve.py

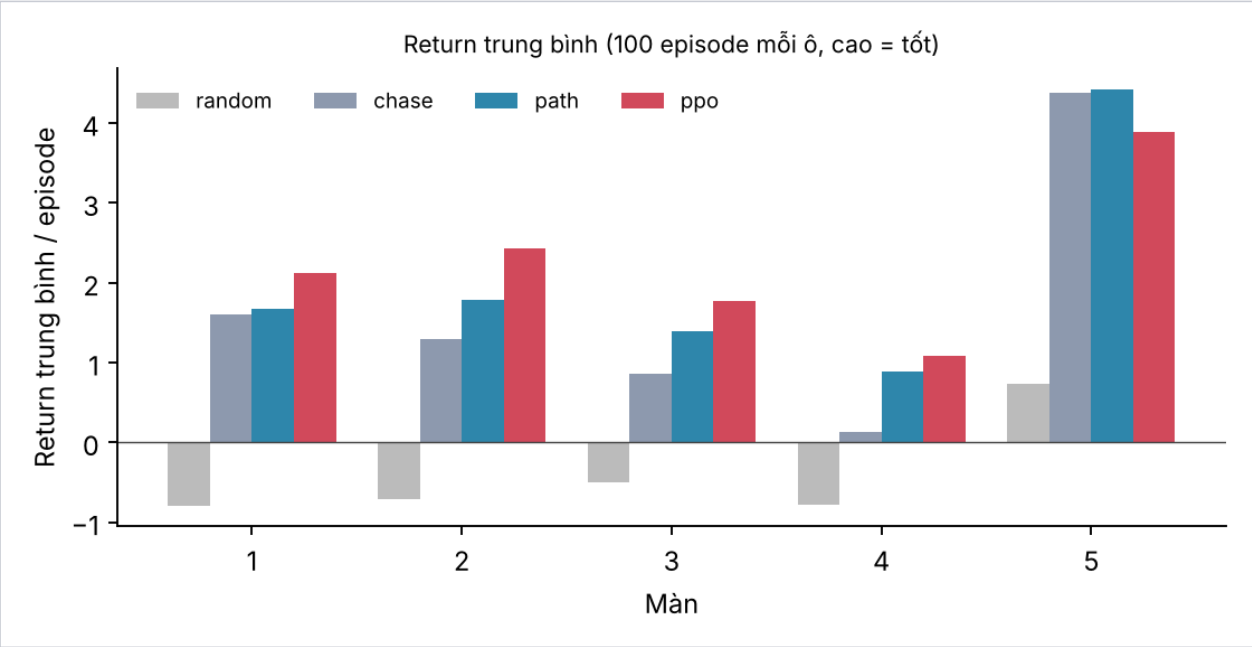


- **Màn 1** (3 seed): return ban đầu -0.44, cuối 1.92 (min 1.85, max 2.03); path 1.67, chase 1.61, random -0.76. Trung bình các seed vượt mốc path từ khoảng 40 nghìn bước.
- **Màn 2** (3 seed): return ban đầu -0.52, cuối 2.07 (min 1.97, max 2.12); path 1.78, chase 1.29, random -0.68. Trung bình các seed vượt mốc path từ khoảng 143 nghìn bước.
- **Màn 4** (3 seed): return ban đầu -0.82, cuối 0.95 (min 0.79, max 1.04); path 0.89, chase 0.14, random -0.66. Trung bình các seed vượt mốc path từ khoảng 184 nghìn bước.

Cách đọc: trục dọc là return trung bình của policy (deterministic) trên 100 episode cố định; đường mảnh là từng seed, đường đậm là trung bình; đường ngang đứt là baseline không học, đo trên cùng 100 episode. Huấn luyện từ đầu (không curriculum) với cùng PPO_KW như train.py, không dùng model trong game.

Kết quả 1: PPO so với baseline

Người chơi giả viết tay · eval_current.csv, 100 episode mỗi ô



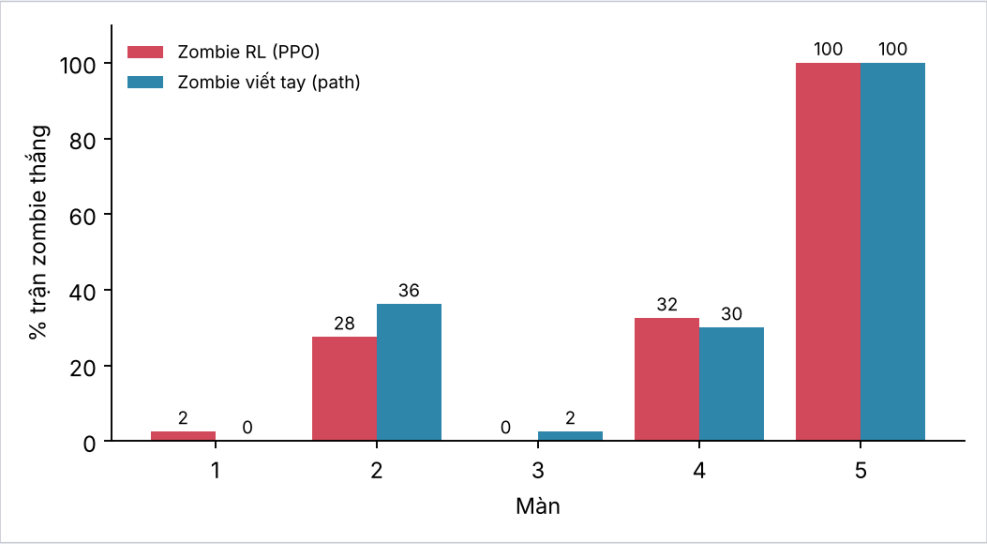
Màn	return random	return chase	return path	return ppo	catch PPO	catch path
1	-0.79	1.61	1.68	2.12	69%	54%
2	-0.70	1.29	1.78	2.43	58%	39%
3	-0.50	0.86	1.39	1.78	31%	22%
4	-0.77	0.14	0.89	1.08	10%	5%
5	0.73	4.37	4.42	3.89	95%	93%

Cách đọc: return = tổng reward trong một episode, trung bình 100 episode. Return cao hơn nghĩa là policy tích lũy reward tốt hơn *theo đúng hàm reward đã thiết kế*; nên phải kiểm chứng thêm bằng tỉ lệ thắng trước đối thủ độc lập.

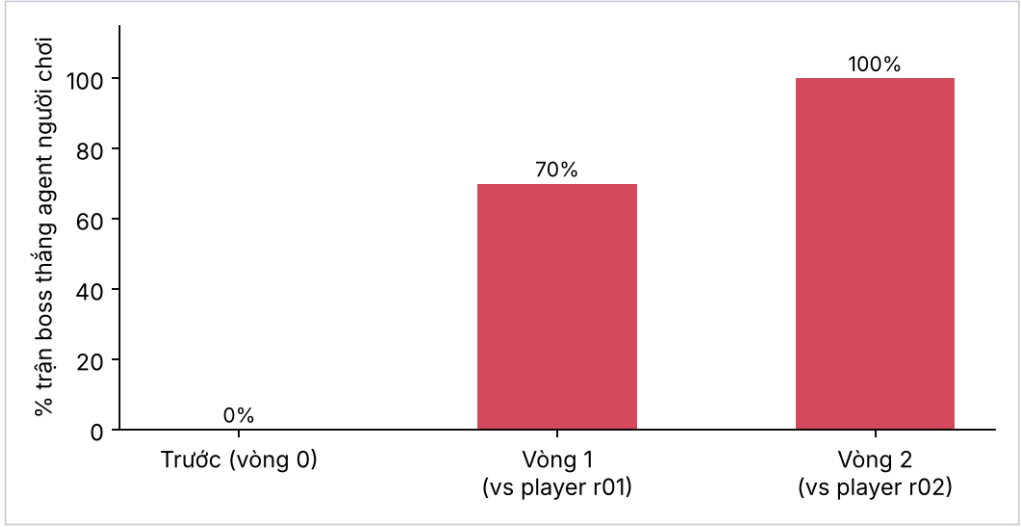
- **Return của PPO cao nhất trong 4 chính sách ở 4/5 màn**; cao hơn path ở màn 1, 2, 3, 4.
- **Bước bị tường chặn** (màn 1): PPO 6%, path 15%, chase 31%. PPO ít "đập vào tường" nhất, nhất quán với reward theo đường đi thật.
- Màn 5 (boss): "catch" nghĩa là hạ được người chơi giả trong trận tay đôi.
- **Giới hạn**: đây là so sánh với người chơi giả viết tay và với số seed huấn luyện = 1 cho mỗi màn. Phép đo độc lập hơn ở trang 18.

Kết quả 2: trước agent người chơi RL

Số liệu từ league.csv và match_eval.csv



Vòng 2 (người chơi r02): % trận zombie thắng, 80 trận mỗi màn, so với zombie đi theo đường trên cùng trận.



Boss: tỉ lệ thắng trước agent người chơi. 0% (trước), 70% (vòng 1), 100% (vòng 2).

100 lượt chơi cả game (1 mạng)	Zombie thắng	95% CI	So với PPO (McNemar)
Zombie PPO	22 / 100	15–31%	-
Zombie đi theo đường (path)	17 / 100	11–26%	p = 0.472, chưa khác biệt
Zombie đi thẳng (chase)	3 / 100	1–8%	p < 0.001, khác biệt

- **Có ý nghĩa thống kê**: zombie PPO thắng nhiều hơn zombie đi thẳng trong chế độ campaign (22 so với 3, p < 0.001).
- **Chưa có bằng chứng** PPO hơn zombie đi theo đường: 22 so với 17, p = 0.47; mọi màn đều p > 0.3. Khoảng tin cậy chồng nhau.
- **Boss** là chỗ khác biệt rõ nhất về mặt hành vi: từ 0% lên 100% sau khi thêm action giữ đòn và phạt vung trượt (trang 19).

Chế độ level (số trận zombie thắng / 100)

Màn (100 trận)	PPO	path	chase	p (PPO vs path)
1	0	1	0	1.00
2	8	13	1	0.38
3	0	0	0	1.00
4	8	7	4	1.00
5	0	0	0	1.00

Số liệu level/campaign là phép đo ban đầu trước self-play (match_eval.csv); số ở biểu đồ phía trên là từ league.csv sau vòng 2, nên không so trực tiếp con số giữa hai nguồn.

Những hiện tượng RL đã gặp và cách xử lý

Chẩn đoán dựa trên hành vi quan sát được

Hiện tượng quan sát được	Chẩn đoán (khái niệm RL)	Biện pháp	Kết quả
Zombie lao đầu vào tường vẫn được điểm	Reward chưa đúng mục tiêu (misspecified): khoảng cách thẳng không phản ánh tiến độ thật	Đo quãng đường đi thật bằng Dijkstra 8 hướng (không cắt góc)	Bước bị tường chặn ở màn 1: PPO 6% so với chase 31%
Boss bị agent nhử: vung tay rồi lùi đúng nhịp thì không bao giờ trúng; boss thua 100/100 trận	Đối thủ khai thác độ trễ cố định của đòn đánh; action space không cho boss cách phản ứng (policy chỉ chọn hướng)	Thêm action giữ đòn (9 → 18), cho zombie nhích theo người chơi khi vung (lunge), phạt vung trượt −0.05	Tỉ lệ boss thắng: 0% → 70% (vòng 1) → 100% (vòng 2)
Đổi số action làm model cũ không nạp được	Kích thước lớp ra khác nhau	Warm-start: sao chép trọng số, bias giữ đòn −1.5	Không phải huấn luyện lại từ đầu
Bản zombie mới "thắng hơn" nhưng do ngẫu nhiên	Phương sai của phép đánh giá	Cùng seed, cùng đối thủ; chỉ giữ khi hơn ≥ 0.03	Loại 2 bản kém hơn ở vòng 2 (màn 2: 36.7% → 23.3%; màn 4: 25.8% → 24.2%)
Train nhiều env song song bị chậm	Các tiến trình tranh nhau nhân CPU (kỹ thuật, không phải RL)	OMP_NUM_THREADS=1 và torch.set_num_threads(1) mỗi env	Tốc độ ổn định ≈ 3 000 bước/giây

Bài học: reward và action space quyết định chính sách học được nhiều hơn việc chỉnh siêu tham số. Cả hai lỗi lớn đều chỉ lộ ra khi cho một đối thủ biết học tấn công chính sách, không lộ ra khi đo return trong môi trường huấn luyện.

Hạn chế và hướng phát triển (góc nhìn RL)

Nói rõ giới hạn của kết quả

Hạn chế hiện tại

- **Hiệu quả so với baseline mạnh còn hạn chế**: PPO ngang zombie đi theo đường ở các màn thường (p = 0.47 ở campaign); chỉ boss có cải thiện rõ.
- **POMDP không bộ nhớ**: policy truyền thẳng, không thấy đồng đội, nên chưa học được phối hợp vây bắt.
- **Ít lần lặp**: league mới chạy 2 vòng, mỗi cấu hình một seed; đường học nhiều seed chỉ có cho 3 màn.
- **Chưa có ablation**: chưa đo riêng đóng góp của curriculum, replay, từng thành phần reward, action giữ đòn.
- **Chưa tìm siêu tham số**: dùng mặc định SB3 cộng vài giá trị chọn tay.
- **Chưa thử với người chơi thật**; agent người chơi có thể có điểm yếu khác con người.

Hướng phát triển

- **Thêm vị trí đồng đội vào observation** hoặc dùng huấn luyện tập trung, thực thi phân tán (CTDE, ví dụ MAPPO) để zombie học phối hợp.
- **Policy có bộ nhớ** (LSTM hoặc xếp chồng vài khung hình) để suy ra ý định người chơi từ quá khứ.
- **Chạy nhiều seed và nhiều vòng league**, báo cáo trung bình và khoảng tin cậy; tăng số trận đánh giá.
- **Ablation có hệ thống**: bỏ curriculum, bỏ action giữ đòn, bỏ shaping, đo từng thay đổi.
- **Thu log của người chơi thật** (thư mục playtest_logs) rồi đánh giá zombie trước hành vi đó.
- **Cân bằng độ khó boss** bằng cách chỉnh máu/damage trong config.py (không cần huấn luyện lại).

Đối chiếu khái niệm RL với dự án

Khái niệm RL	Trong dự án	Xem
Tác tử (agent)	Zombie / boss (policy chung mỗi màn); người chơi	trang 5
Môi trường (environment)	game_core.World; env_zombie.py, zombie_world_env.py, player_env.py (Gymnasium)	trang 11
Trạng thái / quan sát (state / observation)	29 số (zombie), 113 số (người chơi); quan sát một phần (POMDP)	trang 7
Không gian hành động	Discrete(18); MultiDiscrete([3, 3, 9, 2, 2, 2])	trang 7
Reward, return, chiết khấu γ	Bảng reward; γ = 0.99, bước 0.1 s	trang 8
Reward shaping	0.3 × (d trước − d sau), d = đường đi thật	trang 8
Policy, hàm giá (value function)	MLP 64×64 (zombie), 256×256 (người chơi); nhánh actor và critic riêng	trang 10
Actor–critic, on-policy, PPO, clip	PPO của stable-baselines3, ε = 0.2	trang 9
GAE, advantage	γ = 0.99, λ = 0.95, chuẩn hóa advantage	trang 9
Khám phá (exploration), entropy	Lấy mẫu ngẫu nhiên khi học; hệ số entropy 0.01	trang 9
Terminated và truncated	Hết giờ là truncated, SB3 bootstrap V	trang 6
Chia sẻ tham số (parameter sharing)	Mọi zombie của một màn dùng chung một mạng	trang 7
Curriculum, transfer, replay	Màn k học tiếp từ màn k−1; 50% episode ở bản đồ cũ	trang 12
Warm-start khi đổi action space	Sao chép trọng số, bias giữ đòn −1.5	trang 12
Self-play, tính không dừng (non-stationarity)	Đóng băng đối thủ mỗi pha; nhóm đối thủ; ngưỡng giữ bản 0.03	trang 14
Reward hacking	Lao vào tường; nhử đòn boss	trang 19
Đánh giá: baseline, seed, CI, McNemar	random / chase / path; cùng seed; Wilson 95%	trang 15

Từ vựng RL dùng trong tài liệu

Định nghĩa ngắn

Tác tử (agent)	Thực thể quan sát môi trường, chọn hành động và học từ phần thưởng.
Môi trường (environment)	Mọi thứ ngoài tác tử: nhận hành động, trả về quan sát và phần thưởng mới.
Quan sát (observation)	Thông tin tác tử nhận được tại một bước; có thể chỉ là một phần của trạng thái thật.
Hành động (action)	Lựa chọn của tác tử ở mỗi bước.
Phần thưởng (reward)	Số thực môi trường trả về sau mỗi bước, là tín hiệu duy nhất cho biết hành vi tốt hay xấu.
Return	Tổng reward được chiết khấu từ một thời điểm đến hết episode.
Hệ số chiết khấu γ	Trọng số giảm dần cho reward ở xa; γ gần 1 nghĩa là nhìn xa hơn.
Policy π(a | o)	Phân phối xác suất chọn hành động a khi quan sát o; ở đây là một mạng nơ-ron.
Hàm giá V(o)	Ước lượng return kỳ vọng từ quan sát o; dùng làm đường cơ sở (baseline) giảm phương sai.
Lợi thế (advantage)	Hành động tốt hơn mức trung bình bao nhiêu: A = Q − V. Ước lượng bằng GAE.
Episode	Một lượt tương tác từ trạng thái đầu đến khi kết thúc hoặc bị cắt.
Terminated / truncated	Kết thúc thật (thắng, thua, chết) / bị cắt do hết giờ; truncated cần bootstrap giá trị.

On-policy	Dữ liệu dùng để cập nhật được thu bằng chính policy hiện tại (PPO), không dùng lại dữ liệu cũ.
Khám phá (exploration)	Thử những hành động chưa chắc tốt để tìm hành vi tốt hơn; ở đây bằng lấy mẫu và entropy.
Entropy	Độ ngẫu nhiên của policy; thưởng entropy để tránh policy quá tất định quá sớm.
Clipping (PPO)	Giới hạn tỉ số ρ trong [1−ε, 1+ε] để mỗi lần cập nhật không đổi policy quá xa.
GAE	Cách ước lượng advantage kết hợp nhiều bước với tham số λ, cân bằng độ chệch và phương sai.
Reward shaping	Thêm reward phụ để học nhanh hơn; dùng sai có thể làm đổi mục tiêu thật.
POMDP	MDP trong đó tác tử chỉ thấy quan sát một phần của trạng thái.
Curriculum learning	Huấn luyện từ nhiệm vụ dễ đến khó, thường kế thừa policy của bước trước.
Self-play	Tác tử học bằng cách đấu với (các phiên bản của) chính hệ thống học.
Baseline	Phương pháp không học dùng làm mốc so sánh (random, chase, path).
Seed	Giá trị khởi tạo bộ sinh số ngẫu nhiên, để lặp lại đúng một trận hoặc lần huấn luyện.

Tài liệu tham khảo

Nguồn của các khái niệm và công cụ dùng trong dự án

[1] Sutton, R. S., Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.

[2] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.

[3] Schulman, J., Moritz, P., Levine, S., Jordan, M., Abbeel, P. (2016). High-Dimensional Continuous Control Using Generalized Advantage Estimation. ICLR. arXiv:1506.02438.

[4] Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., Dormann, N. (2021). Stable-Baselines3: Reliable Reinforcement Learning Implementations. *JMLR* 22(268).

[5] Towers, M. et al. (2024). Gymnasium: A Standard Interface for Reinforcement Learning Environments. arXiv:2407.17032.

[6] Ng, A. Y., Harada, D., Russell, S. (1999). Policy invariance under reward transformations: Theory and application to reward shaping. ICML.

[7] Bengio, Y., Louradour, J., Collobert, R., Weston, J. (2009). Curriculum learning. ICML.

[8] Vinyals, O. et al. (2019). Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature* 575, 350–354.

[9] Kaelbling, L. P., Littman, M. L., Cassandra, A. R. (1998). Planning and acting in partially observable stochastic domains. *Artificial Intelligence* 101, 99–134.

[10] Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. *JASA* 22(158), 209–212.

[11] McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika* 12(2), 153–157.

Các màn chơi

Màn	Tên	Bản đồ	Đợt zombie	Súng
1	Kho chứa	12×16	3 đợt: 3 vừa · 4 vừa · 3 vừa + 1 mập	Súng lục
2	Hành lang	14×18	4 đợt, bắt đầu có zombie ốm	Súng lục
3	Tầng hầm	16×20	4 đợt trộn đủ 3 loại	Súng lục, M4A1
4	Phòng thí nghiệm	18×20	5 đợt, đợt cuối 3 mập + 3 ốm + 3 vừa	Súng lục, M4A1
5	Boss	18×18	Chỉ người chơi và boss 1500 HP	Súng lục, M4A1

Luật thắng: dọn hết các đợt zombie thì qua màn; hạ boss ở màn 5 thì game kết thúc luôn. M4A1 mở khóa sau khi qua màn 2, tức từ màn 3.



Thứ tự này cũng là thứ tự huấn luyện (curriculum, trang 12).

Ảnh minh họa các màn nằm ở trang 4.

Bốn loại kẻ địch

Loại	Máu	Damage	Tốc độ (ô/s)	Tầm đánh	Vung tay → ra đòn
Mập (heavy)	200	25	1.2	0.75	0.55 s
Vừa (vest)	100	10	2.4	0.65	0.40 s
Ốm (skinny)	50	10	4.3	0.60	0.30 s
Boss	1500	35	3.0 (pha 2 nhanh hơn)	0.90	0.40 s + lao tới

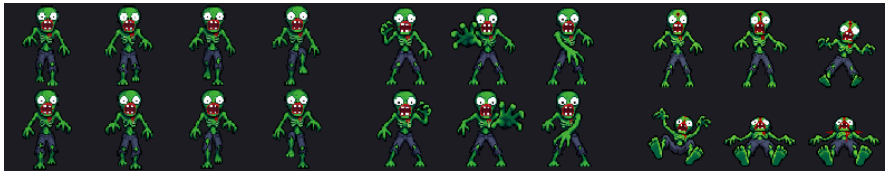
Pha 2 của boss: dưới 50% máu, boss nhanh hơn và biết lao tới khi ở xa. **Sprite** là bảng ảnh do nhóm cung cấp, được cắt lưới, xóa nhiễu và căn chân.



Mập: walk · attack · death



Vừa: walk · attack · death



Ốm: walk · attack · death



Boss: walk · attack · death

Hai khẩu súng



Súng lục: hàng trên là bắn, hàng dưới là nạp đạn (6 frame mỗi hàng)



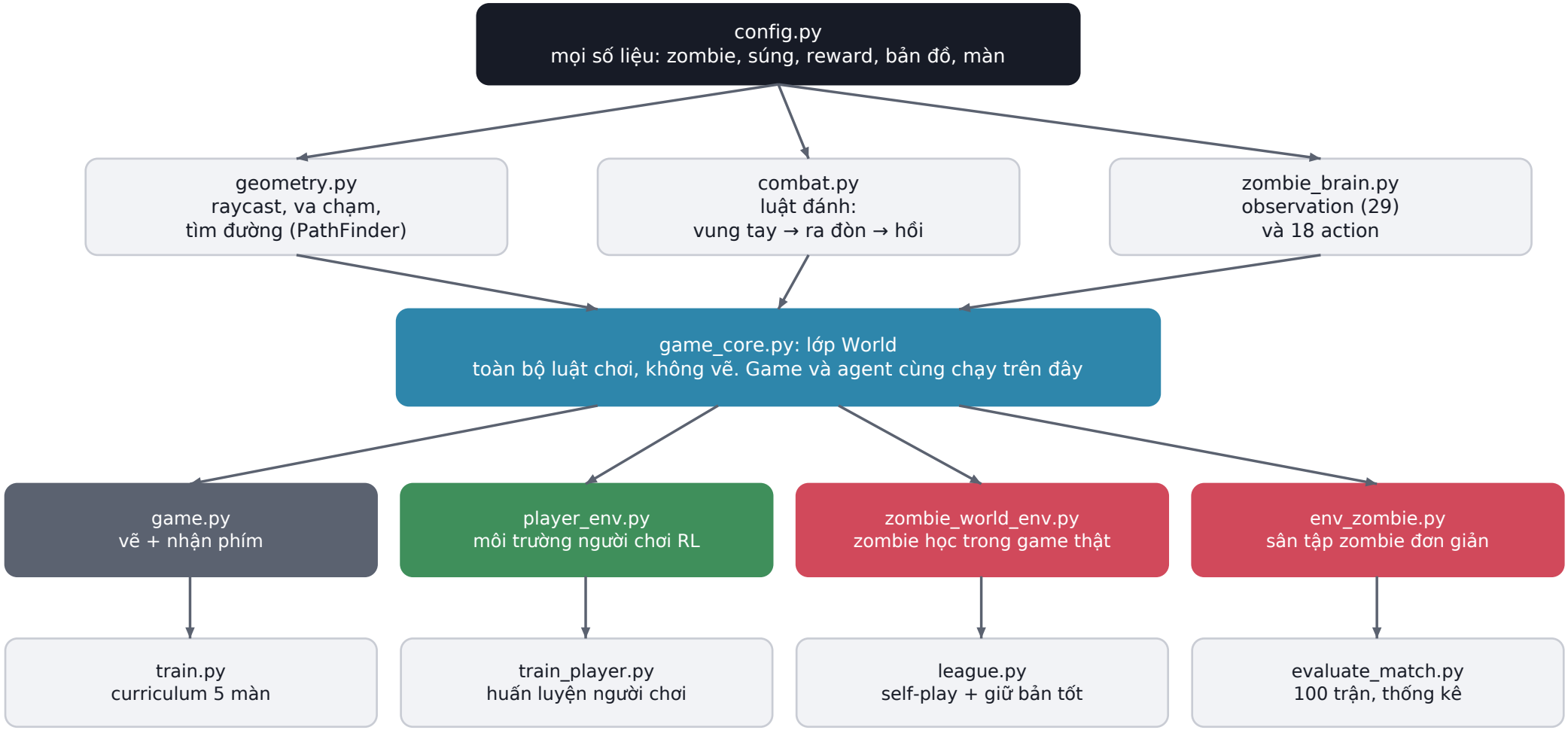
M4A1: hàng trên là bắn, hàng dưới là nạp đạn (6 frame mỗi hàng)

Súng	Damage	Tốc độ bắn	Băng đạn	Nạp đạn	Mở khóa	Tầm
Súng lục	25	0.28 s/phát	12	1.2 s	Màn 1	10 ô
M4A1	22	0.12 s/phát	35	1.6 s	Màn 3	12 ô

- Súng lục có damage 25 để đúng 2 phát hạ được zombie ốm (50 máu).
- **Làm sạch ảnh** (tools/prepare_weapons.py): xóa nền bằng xói mòn rồi flood-fill (để không ăn vào thân súng tối), xóa đốm lẻ, căn đáy các frame, khớp tư thế đầu và cuối của animation nạp đạn.
- Đạn trúng tức thì theo phương ngang (hitscan), người chơi chỉ quay trái/phải nên ngắm là bài toán một chiều.

Kiến trúc code

Một bộ luật dùng chung cho game, huấn luyện và đánh giá



Điểm mấu chốt: *game_core.py* không biết gì về cửa sổ hay đồ họa. Vì vậy cùng một bộ luật chạy được trong game (có vẽ, 60 FPS) lẫn trong huấn luyện hàng triệu bước (không vẽ). Observation và action của zombie nằm trong *zombie_brain.py* nên policy thấy cùng một thế giới khi học và khi chơi thật.

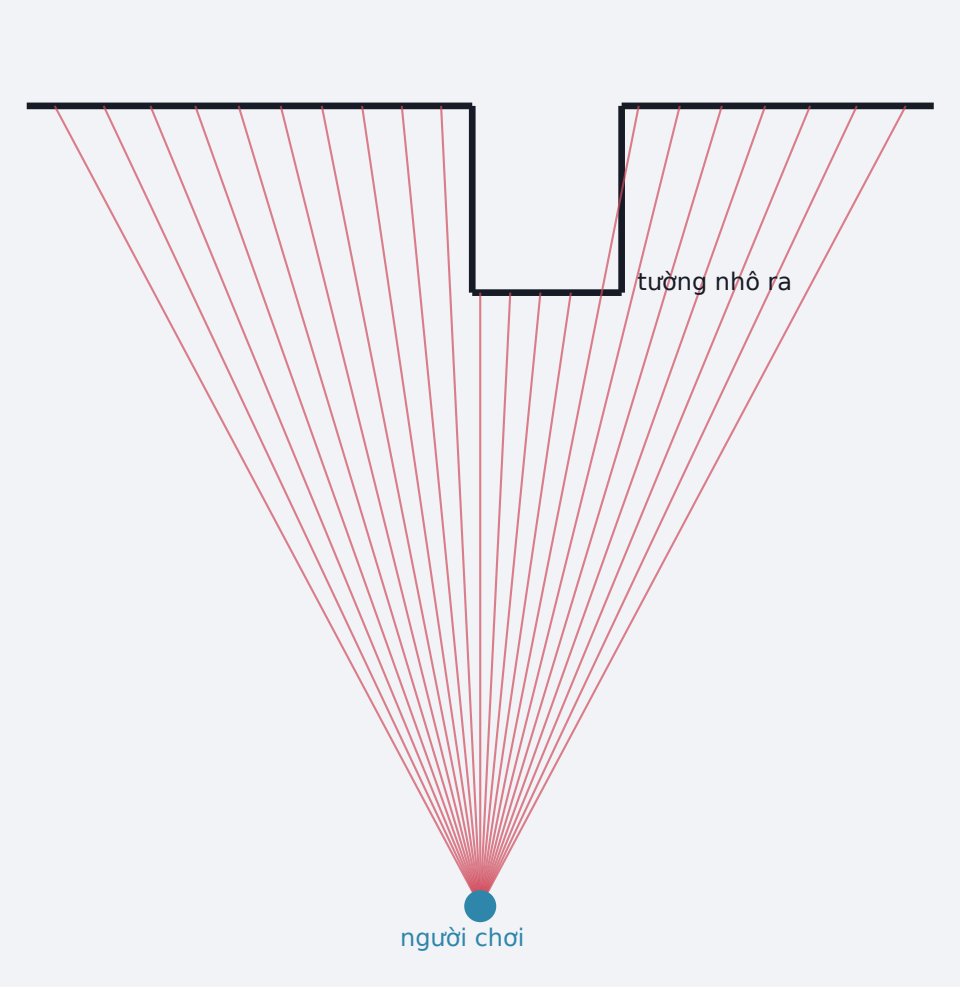
Chức năng từng file

Số dòng tính theo file trong thư mục dự án

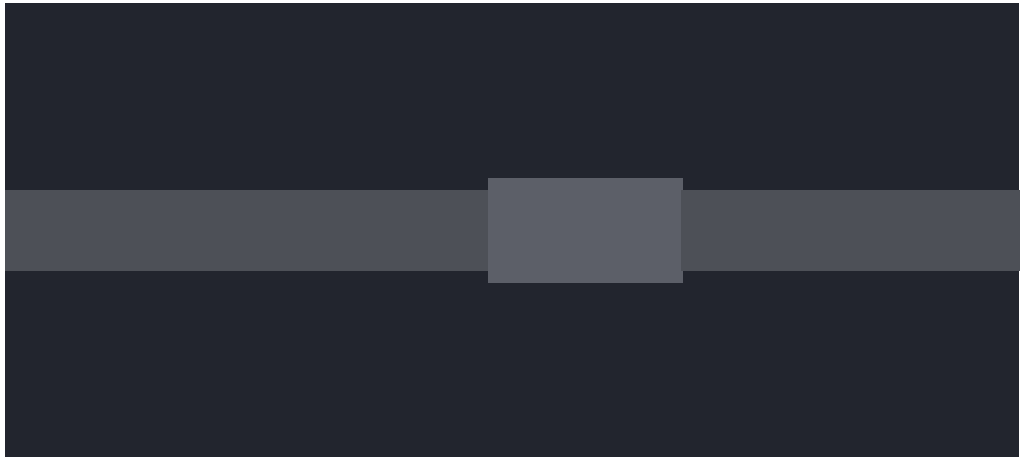
File	Dòng	Làm gì
config.py	203	Mọi hằng số: 4 loại zombie, 2 súng, reward, 5 bản đồ, đợt zombie, độ giỏi người chơi giả
geometry.py	209	parse_map, cast_ray (DDA), move_with_collision, clear_path, line_of_sight, in_fire_line, PathFinder (Dijkstra 8 hướng), resolve_shot
combat.py	75	AttackState (windup → strike → recovery → cooldown), BossMods (pha 2, lao tới)
zombie_brain.py	92	ACTIONS (9 hướng × 2 chế độ đánh = 18), build_obs (29 số), baseline_action (path / chase)
game_core.py	379	World, Player, Zombie, Controls, ModelBank: luật chơi, đợt zombie, súng, nạp đạn, điều khiển zombie
game.py	459	Vẽ tường bằng raycasting, sprite zombie, súng, HUD, minimap, bảng RL; xử lý phím và chuột
sprites.py	201	Cắt sprite sheet, tô màu theo loại, animation súng
env_zombie.py	163	Gymnasium env cho zombie học (một zombie, người chơi giả)
player_bot.py	125	Người chơi giả viết tay: phản xạ, nhiễu ngắm, thả diều quanh cột
train.py	140	Huấn luyện PPO theo curriculum, so sánh với 3 baseline (random / chase / path)
player_env.py	240	Env cho agent người chơi (113 số quan sát), ScriptedPlayer, PlayerAgentDriver
train_player.py	55	Huấn luyện agent người chơi bằng PPO
zombie_world_env.py	123	Zombie học đối kháng ngay trong World với agent người chơi
league.py	233	Vòng self-play: zombie học → giữ bản tốt → người chơi học lại → đo
evaluate_match.py	201	Chạy N trận, Wilson CI, kiểm định McNemar, ghi match_eval.csv
tools/learning_curve.py	74	Huấn luyện lại từ đầu và ghi đường học (trang 16)

Tổng cộng 2898 dòng Python ở thư mục gốc.

Vẽ 3D chỉ bằng 2D: raycasting



Nhìn từ trên xuống: mỗi tia đỏ là một cột pixel



Ảnh người chơi thấy: tường càng gần thì cột càng cao

- **Cách làm**: với mỗi cột pixel của màn hình (320 cột), bắn một tia từ người chơi theo góc nhìn (FOV ≈ 66°), dùng thuật toán DDA nhảy từng ô lưới cho tới khi chạm tường.
- Chiều cao cột = hằng số / khoảng cách *vuông góc* (không dùng khoảng cách thẳng, để tường không bị cong).
- Mỗi cột lưu vào **zbuffer**; sprite zombie vẽ sau và chỉ tô những cột mà zombie gần hơn tường.

Cùng hàm *cast_ray* được zombie dùng để "nhìn" 8 hướng quanh mình (một phần của observation, trang 7).

Luật đánh của zombie

combat.py

Khi người chơi lọt vào tầm, zombie không gây damage ngay mà phải **vung tay** (windup), rồi **ra đòn** (strike), rồi **hồi chiêu**. Người chơi có thời gian windup để lùi ra khỏi tầm. Luật này là một phần của môi trường RL: policy phải học thời điểm vung.




```
def update(self, dt, dist, los, want=True):
    self.cooldown -= dt; self.whiffed = False
    if not self.active:
        if want and dist < self.st["attack_range"] and self.cooldown <= 0:
            self.active, self.t, self.struck = True, 0.0, False
        else:
            return False, 0.0
    self.t += dt; damage = 0.0
    if not self.struck and self.t >= self.st["windup"]:
        self.struck = True
        if dist < self.st["attack_range"] + 0.3 and los:
            damage = float(self.st["damage"])
        else:
            self.whiffed = True              # whiff
    if self.t >= self.st["windup"] + RECOVER:
        self.active, self.cooldown = False, self.st["attack_cooldown"]
    return True, damage
```

Liên hệ với RL

- Tham số *want* là bit "cho phép đánh / giữ đòn" của action PPO (trang 7).
- Cờ *whiffed* bật đúng bước vung trượt, để môi trường cộng phạt −0.05 (trang 8).
- Khi đang vung, zombie vẫn nhích theo người chơi (lunge), nên lùi đúng nhịp không còn né được (trang 19).

Cách chạy và gợi ý trình bày

Lệnh chạy (trong D:\Zombie RL, đã kích hoạt .venv)

`python game.py`	Chơi từ màn 1 (zombie PPO)
`python game.py --level 5`	Vào thẳng màn boss
`python game.py --zombies path`	Zombie viết tay, để so cảm giác
`python game.py --watch-agent`	Xem agent người chơi tự chơi
`python train.py --scale 0.1`	Huấn luyện thử nhanh (10% số bước)
`python train_player.py`	Huấn luyện agent người chơi
`python evaluate_match.py`	100 trận mỗi màn, có thống kê
`python league.py --rounds 3`	Self-play nhiều vòng, chạy qua đêm
`python tools/learning_curve.py --level 1`	Huấn luyện từ đầu, ghi đường học

Gợi ý trình bày 10 phút

Phút	Nội dung	Trang
0–1	Bài toán và câu hỏi RQ1–RQ3	3
1–3	Demo game, mở bảng RL (Tab)	4
3–5	MDP, observation, action, reward	6–8
5–6	PPO và siêu tham số	9–10
6–7	Curriculum và self-play	12, 14
7–9	Đánh giá và kết quả, nói đúng giới hạn	15–18
9–10	Hạn chế, hỏi đáp	20

Câu hỏi có thể gặp: Vì sao dùng PPO (trang 9)? Reward có bị khai thác không (trang 19)? Vì sao 100 trận chưa đủ để kết luận (trang 18)? Policy có nhớ quá khứ không (trang 7)? Thuật ngữ nào chưa rõ (trang 22)?

Nhóm thực hiện
Nguyễn Võ Trung Hưng (SE193635) · Ngô Phúc Nguyên (SE192516) · Lê Như Đạt (SE192807)